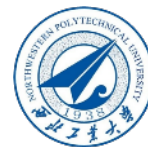


函数



- ◆ 掌握函数的定义与调用方法
- ◆ 掌握函数参数与返回值
- ◆ 掌握变量的作用域
- ◆ 掌握lambda表达式
- ◆ 掌握递归函数

- 函数是组织好的，可重复使用的，用来实现单一或相关功能的代码段。
- 函数能提高应用的模块性和代码的重复利用率。
- 通过前面几章的学习，我们已经了解了很多Python内置函数，通过使用这些内置函数可给我们的编程带来很多便利，提高了我们开发程序的效率。
- 除了使用Python内置函数，也可以根据实际需要定义符合我们要求的函数，这被叫做用户自定义函数。



在python中定义函数的语法如下所示:

```
def 函数名([参数列表]):
```

```
    函数体
```

(1) 函数代码块以def 关键词开头，之后是函数名，def和函数名中间要敲一个空格。

(2) 圆括号内用于定义函数参数，称为形式参数或简称为形参，参数是可选的，函数可以没有参数。如果有多个参数，参数之间用逗号隔开。参数就像一个占位符，当调用函数时，就会将一个值传递给参数，这个值被称为实际参数或实参。在Python中，函数形参不需要声明其类型。

(3) 函数体，指定函数应当完成什么操作，是由语句组成，要有缩进。



在python中定义函数的语法如下所示：

```
def 函数名([参数列表]):
```

```
    函数体
```

(4) 如果函数执行完之后有返回值，称为带返回值的函数，函数也可以没有返回值。带有返回值的函数，需要使用以关键字**return**开头的返回语句来返回一个值，执行**return**语句意味着函数执行的终止。

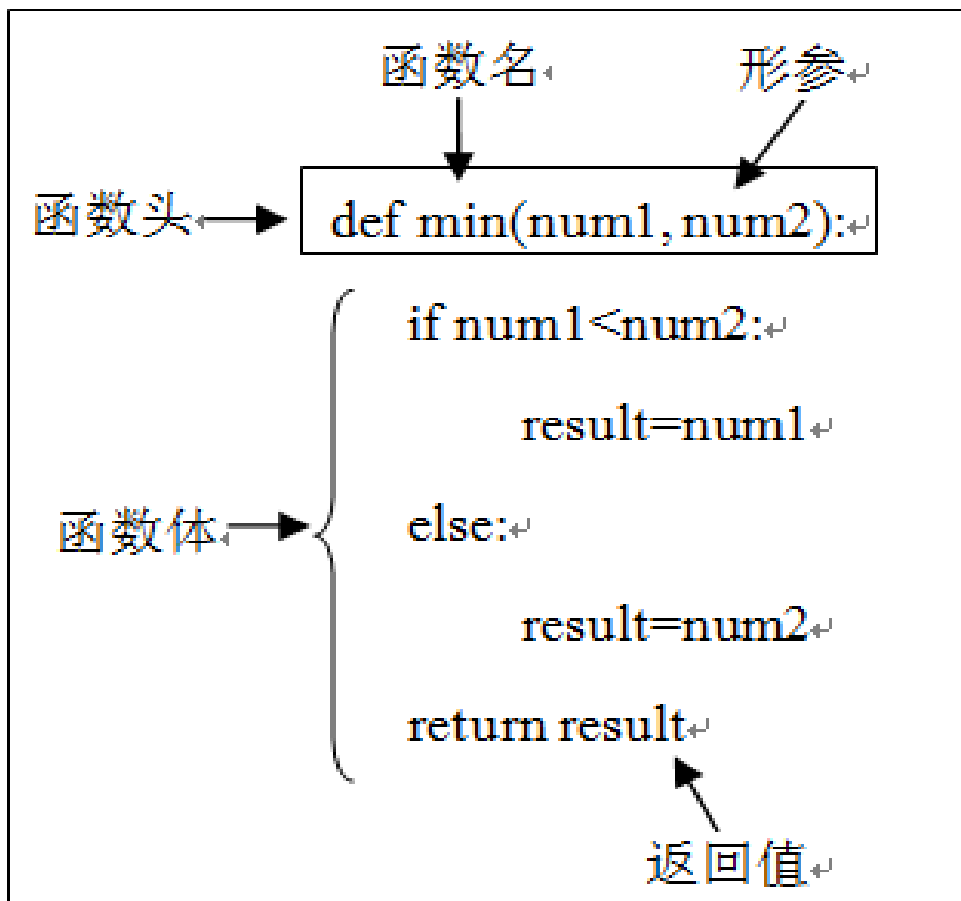
(5) 在定义函数时，开头部分的注释通常描述函数的功能和参数的相关说明，但这些注释并不是定义函数时必需的，可以使用内置函数**help()**来查看函数开头部分的注释内容。

- 自定义函数使用def关键字来标明函数定义的开始。
 - 函数形参不需要声明其类型，也不需要指定函数返回值类型；
 - 即使该函数不需要接收任何参数，也必须保留一对空的圆括号；
 - 括号后面的冒号必不可少；
 - 函数体相对于def关键字必须保持一定的空格缩进。

● 怎样定义函数



下面定义一个找出两个数中较小的函数。这个函数被命名为min，它有两个参数：num1和num2，函数返回这两个数中较小的那个。



- 在函数定义中，定义了函数的功能，即定义了函数要执行的操作。
- 要使函数发挥功能，必须调用函数，调用函数的程序被称为调用者。调用函数的方式是函数名（实参列表），实参列表中的参数个数要与形参个数相同，参数类型也要一致。
- 当程序调用一个函数时，程序的控制权就会转移到被调用的函数上。当执行完函数的返回值语句或执行到函数结束时，被调用函数就会将程序控制权交还给调用者。根据函数是否有返回值，调用函数有两种方式。

根据函数是否有返回值，调用函数有两种方式。

(1) 带有返回值的函数调用

对这种函数的调用通常当作一个值处理，例如：

```
smaller = min(2, 3)
```

```
print(min(2, 3))
```

(2) 没有返回值的函数调用

如果函数没有返回值，对函数的调用是通过将函数调用当作一条语句来实现的。

调用函数例子：

```
In [133]: ▶ def printStr(str1) :  
            print(str1)      #打印任何传入的字符串  
            printStr('hello world')  
  
            hello world
```

调用函数例子：

```
In [134]: ▶ def sum(num1, num2):  
            result = 0  
            for i in range(num1, num2 + 1):  
                result += i  
            return result  
  
            :  
            "Sum from 1 to 10 is", sum(1, 10))  
            "Sum from 11 to 20 is", sum(11, 20))  
            "Sum from 21 to 30 is", sum(21, 30))  
  
            Call the main function  
  
            to 10 is 55  
            1 to 20 is 155  
            21 to 30 is 255
```

这个程序包含sum函数和main函数，在Python中main也可以写成其它任何合适的标识符。程序脚本在最后一行调用main函数。习惯上，程序里通常定义一个包含程序主要功能的名为main的函数。

● 函数调用



调用函数例子：

```
1.  def sum(num1, num2):
2.      result = 0
3.      for i in range(num1, num2 + 1):
4.          result += i
5.      return result
6.  def main():
7.      print("Sum from 1 to 10 is", sum(1, 10))
8.      print("Sum from 11 to 20 is", sum(11, 20))
9.      print("Sum from 21 to 30 is", sum(21, 30))
10. main() # Call the main function
```

解释器从第一行开始一行一行地读取程序语句；读到第1行的函数头时，将函数以及函数体（第1到5行）存储在内存中。然后，解释器将main函数的定义（第6到9行）读取到内存。最后，解释器读取到第10行时，调用main函数，即main函数被执行。

注意：这里的main函数是定义在sum函数之后。在Python中，函数在内存中被调用，在调用某个函数之前，该函数必须已经调入内存，否则系统会出现要调用的函数未调入的错误。

```
In [134]: ▶ def sum(num1, num2):  
            result = 0  
            for i in range(num1, num2 + 1):  
                result += i  
            return result  
def main():  
    print("Sum from 1 to 10 is", sum(1, 10))  
    print("Sum from 11 to 20 is", sum(11, 20))  
    print("Sum from 21 to 30 is", sum(21, 30))  
main() # Call the main function
```

```
Sum from 1 to 10 is 55  
Sum from 11 to 20 is 155  
Sum from 21 to 30 is 255
```

- 声明函数时建立的参数为形式参数，简称形参；
- 调用函数时，提供函数需要的参数的值，即实际参数，简称实参。
- 实际参数值默认按**位置顺序**依次传递给形式参数，如果参数个数不对，则会产生错误。

- 在定义函数时，如果有多个形参，则需要使用逗号进行分隔。同时，对于绝大多数情况，在函数内部直接修改形参的值不会影响实参。

```
In [140]: ▶ def plus(a, b):  
            print(a)  
            a += b  
            print(a)  
a, b = 3, 2  
plus(a, b)  
print(a, b)
```

```
3  
5  
3 2
```

➤ 例子:

```
In [141]: ► def modify(v):           # 修改列表元素值
            v[0]=v[0]+1
            a = [2]
            modify(a)
            print(a)

[3]
```

```
In [142]: ► def append(v, item):     # 为列表添加元素
            v.append(item)
            append(a, 4)
            print(a)

[3, 4]
```

➤ 例子:

```
In [146]: ► def change(d):           # 修改字典元素的值
            d['age'] = 30
            b = {'name': 'Wang', 'age':20, 'sex': 'Male'}
            change(b)
            print(b)

            {'name': 'Wang', 'age': 30, 'sex': 'Male'}
```

- Python函数参数定义时并不指定参数类型，为减少函数内部数据处理时可能发生的错误，让程序运行更加稳健。
- 可以采用`isinstance()`函数来显示判断参数的类型，一旦用户输入的类型不符合处理要求，进入错误处理逻辑，这样编写的程序会更加严密。

➤ 编写函数计算圆的面积。

```
In [149]: ▶ from math import pi as PI
def CircleArea(r):
    if isinstance(r, int) or isinstance(r, float):
        return PI*r*r
    else:
        return('You must give me an integer or float as radius.')
print(CircleArea(3))

28.274333882308138
```



- Python 中一切都是对象，调用函数时的参数传递就是传可变对象还是传不可变对象。函数调用时传递的参数类型分为可变类型和不可变类型。
- **不可变类型**：若a是数字、字符串、元组这三种类型中的一种，则函数调用fun(a)时，传递的只是a的值，**在fun(a)内部修改 a** 的值，只是修改另一个复制的对象，不会影响a本身。
- **可变类型**：若a是列表、字典这两种类型中的一种，**则函数调用fun(a)时**，传递的是a所指的**对象**，**在fun(a)内部修改a**的值，**fun(a)外部的a也会受影响**。



- 函数的作用就在于它处理参数的能力，当调用函数时，需要将实参传递给形参。函数参数的使用可以分为两个方面，一是函数形参是如何定义的，二是函数在调用时实参是如何传递给形参的。
- 在Python中，定义函数时不需要指定参数的类型，形参的类型完全由调用者传递的实参本身的类型来决定。
- 函数形参的表现形式主要有：**位置参数、关键字参数、默认值参数、可变长度参数。**



1. 位置参数

位置参数是最常用的形式，调用函数时**实参和形参的顺序**以及**必须严格一致**，并且实参和形参的**数量必须相同**。

```
In [27]: ► def menu(food, cigarette, wine):  
           print("主食:", food, "香烟:", cigarette, "白酒:", wine)  
           menu("面条", "红塔山", "郎酒")
```

主食：面条 香烟：红塔山 白酒：郎酒



2. 默认值参数

定义带有默认值参数的函数的语法格式：

`functionName(…, 参数名=默认值)`

在定义函数的时候，为形参设置了默认值。在调用设置了默认值参数的函数时，如果没有给设置了默认值的形式参数传递实参，那么这个形参就将使用默认值。

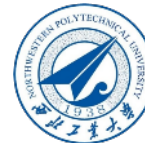
注意：在定义带有默认值参数的函数时，默认值参数必须出现在函数形参列表的最右端，任何一个默认值参数右边都不能再出现非默认值参数。



2. 默认值参数

```
In [174]: ► def add(x=3, y=5):  
            return(x + y)  
            print(add(9))
```

14



3. 关键字参数

关键参数指调用函数时的参数传递方式，与函数定义无关。通过关键参数可以**按照参数名字**传递值，明确指定哪个值传递给哪个参数，**实参的顺序可以和形参的顺序不一致**，并不影响参数值的传递结果。

还可以**混合位置参数和关键参数**，但是要**确保位置参数在关键参数的左边**。

```
In [175]: ▶ def menu(food, cigarette, wine):  
            print("主食:", food, "香烟:", cigarette, "白酒:", wine)  
            menu(wine="茅台", food="米饭", cigarette="好猫")  
            menu("米粉", wine="五粮液", cigarette="云烟")
```

```
主食：米饭 香烟：好猫 白酒：茅台  
主食：米粉 香烟：云烟 白酒：五粮液
```



4. 可变长度参数

定义带有可变长度参数的函数的语法格式如下：

functionName (arg1, *tupleArg, **dictArg)

- tupleArg和dictArg称为可变长度参数。
- tupleArg前面的“*”表示这个参数是一个元组参数，用来接收任意多个实参并将其放在一个元组中。
- dictArg前面的“**”表示这个参数是个字典参数(键值对参数)，用来接收类似于关键字参数一样显示赋值形式的多个实参并将其放入字典中。
- 可以把tupleArg、dictArg看成两个默认参数，调用带有可变长度参数的函数时，多余的非关键字参数放在元组参数tupleArg中，多余的关键字参数放字典参数dictArg中。



4. 可变长度参数

```
In [176]: ► def varLength (arg1,*tupleArg,**dictArg):  
            print("arg1=", arg1)  
            print("tupleArg=", tupleArg)  
            print("dictArg=", dictArg)  
            varLength('hello world','Python','C',a=1,b=2)  
  
            arg1= hello world  
            tupleArg= ('Python', 'C')  
            dictArg= {'a': 1, 'b': 2}
```



4. 可变长度参数

```
In [178]: ► def print_args(arg1, arg2, *pargs, **kargs):  
            print("arg1 is ", arg1)  
            print("arg2 is ", arg2)  
            print("pargs is: ", pargs)  
            print("kargs is:", kargs)  
            print_args("主食", "面条", "白酒", "汾酒", 香烟="芙蓉王")  
  
            arg1 is  主食  
            arg2 is  面条  
            pargs is: ('白酒', '汾酒')  
            kargs is: {'香烟': '芙蓉王'}
```



5. 序列解包参数

- 序列解包参数主要指调用函数时的参数传递方式，与函数定义无关。
- 使用序列解包参数调用的函数通常是一个位置参数函数，序列解包参数由一个“*”和序列连接而成，Python解释器将自动将序列解包成多个元素，并一一传递给各个位置参数。
- 创建列表、元组、集合、字典以及其它可迭代对象，称为“序列打包”，因为值被“打包到序列中”。“序列解包”是指将多个值的序列解开，然后放到变量的序列中。下面举例说明：

```
In [179]: dict1 = {"one":1, "two":2, "three":3}
          x, y, z = dict1
          print(x, y, x)
          #字典解包默认的是解包字典的键

          one two one
```



5. 序列解包参数

问题：如何将字典中的键-值对进行序列解包呢？

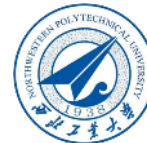
```
In [180]: dict1 = {"one":1,"two":2,"three":3}
          x1,y1,z1 = dict1.items() #用字典对象的items()方法解包字典的“键-值”对
          print(x1,y1,x1)

('one', 1) ('two', 2) ('one', 1)
```

5. 序列解包参数

下面举例说明调用函数时的序列解包参数的用法：

```
In [181]: ▶ def print1(x, y, z):  
            print(x, y, z)  
            tuple1=('姓名', '性别', '籍贯')  
            print1(*tuple1)    ##将tuple1解开成3个元素并分别赋给x, y, z  
  
            姓名 性别 籍贯
```



函数的返回值：

- 函数可以没有返回值。
- `return`语句用来从一个函数中返回并结束函数的执行，同时返回一个任意类型的值。
- 不论`return`语句出现在函数的什么位置，一旦得到执行将直接结束函数的执行。
- 如果函数没有`return`语句或者执行了不返回任何值的`return`语句，即返回空值。
- Python支持同时返回多个值，多个值以元组的形式返回。

函数的返回值:

```
In [1]: ▶ def adddiv(a, b):  
        a, b = a + b, a / b  
        return a, b      #返回一个元组, 包含a和b的值  
add, sub = adddiv(10, 3)  #元组的拆封  
print("参数之和是: %d, 参数相除是: %.2f"%(add, sub))
```

参数之和是: 13, 参数相除是:3.33

● 主函数

- Python没有通常意义上的主函数，在程序中经常会放置以下代码，用以表示主要的代码执行：

`if __name__=='__main__'。`

- `__name__`为系统变量，为当前模块的名称。

- lambda表达式可以用来声明**匿名函数**，也就是**没有函数名字**的**临时使用的函数**。在使用函数作为参数的时候，如果传入的函数比较简单或者使用次数较少，直接定义这些函数就显得比较浪费，这时就可以使用lambda表达式。

lambda <variables>:<expression>

其中variables是函数的参数，expression是函数的返回值，它们之间用冒号:分隔。

● lambda表达式

1. lambda表达式**必须使用lambda关键字**定义。
2. 在lambda**关键字之后，冒号左边的是参数列表**，可以没有参数，也可以有多个参数，由逗号隔开，**冒号右边是该 lambda 表达式的返回值**。
3. lambda表达式**必须是单行的**。
4. lambda表达式可以直接在程序中使用，也可以将这一表达式赋值给一个变量。
5. lambda表达式的**计算结果就是函数的返回值**。

- **lambda表达式**只可以包含一个表达式，**不允许包含选择、循环**等语法结构，该表达式的计算结果可以看作是函数的返回值，不允许包含复合语句，但在表达式中可以调用其他函数。
- **lambda函数**拥有自己的命名空间，且**不能访问自有参数列表之外或全局命名空间里的参数**。

➤ lambda表达式举例:

```
In [2]: ▶ f = lambda x, y, z: max(x, y, z)
        print(f(10, 20, 30))
        g = lambda x: x*2
        print(g(3))
        f = lambda x, y, z: x+y+z    #定义一个lambda表达式, 求三个数的和
        print(f(1, 2, 3))

        30
        6
        6
```

- 假如要编写函数实现计算多项式 $1+2*x+y^2+z*y$ 的值，可以简单的定义一个lambda函数来完成这个功能。

```
In [13]: ► polynomial = lambda x, y, z: 1+2*x+y**2+z*y  
          polynomial(1, 2, 3)
```

```
Out[13]: 13
```

变量起作用的代码范围称为**变量的作用域**。在Python中，使用一个变量时并不需要预先声明它，但在真正使用它之前，它必须被绑定到某个内存对象(也即变量被定义、赋值)，变量名绑定将在当前作用域中引入新的变量，同时屏蔽外层作用域中的同名变量。

● 局部变量与全部变量



- 函数内部定义的变量一般为局部变量。
- 不属于任何函数的变量一般为全局变量。
- 引用局部变量比全局变量速度快，因此在程序设计时应优先考虑使用局部变量。
- 全局变量可能会增加不同函数之间的隐式耦合度，因此在实际编程过程中不适宜大量使用。

- 局部变量只在该函数内起作用，当函数运行结束后，在该函数内部定义的局部变量被自动删除而不可访问。
- 在函数内部定义的全局变量当函数结束以后仍然存在并且可以访问。
- 如果想要在函数内部修改一个定义在函数外的变量值，可以通过`global`来声明或定义，能够同时作用于函数内外。



函数变量的作用域:

In [3]: ▶

```
num = 100
def func():
    global num    #声明num是全局变量
    num = 200    #修改name全局变量的值
    print('在函数内输出num: ', num)
func()
print('在函数外输出num: ', num)
```

在函数内输出num: 200

在函数外输出num: 200

- 不属于任何函数的变量一般为**全局变量**，它们在所有函数之外创建，可以被所有的函数访问，也即模块层次中定义的变量，每一个模块都是一个全局作用域。
- 也就是说，**在模块文件顶层声明的变量具有全局作用域**，模块的全局变量就像是一个模块对象的属性。

- ① 一个变量已在函数外定义，如果在函数内需要修改这个变量的值，并将这个赋值结果反映到函数之外，可以在函数内用**global**声明这个变量为全局变量，明确声明要使用已定义的同名全局变量。
- ② 在函数内部直接使用**global**关键字将一个变量声明为全局变量，即使在函数外没有定义该全局变量，在调用这个函数之后，将自动增加新的全局变量。



➤ 举例:

```
In [4]: ▶ name = ['Chinese', 'Math']    #全局变量
name1 = ['Java', 'Python'] #全局变量
name2 = ['C', 'C++']    #全局变量
def f1():
    name.append('English')    #append方法可改变外部全局变量的值
    print('函数内name: %s' %name)
    name1 = ['Physics', 'Chemistry']
    print('函数内name1: %s' %name1)
    global name2    #如需重新给name2赋值, 需使用global声明全局变量
    name2 = '123'
    print('函数内name2: %s' %name2)
f1()
print('函数外输出name: %s' %name)
print('函数外输出name1: %s' %name1)
print('函数外输出name2: %s' %name2)
```

```
函数内name: ['Chinese', 'Math', 'English']
函数内name1: ['Physics', 'Chemistry']
函数内name2: 123
函数外输出name: ['Chinese', 'Math', 'English']
函数外输出name1: ['Java', 'Python']
函数外输出name2: 123
```

变量的嵌套作用域:

- 嵌套作用域也包含在函数中，嵌套作用域和局部作用域是相对的，嵌套作用域相对于更上层的函数而言也是局部作用域。
- 与局部作用域的区别在于，对一个函数而言，局部作用域是定义在此函数内部的局部作用域，而嵌套作用域是定义在此函数的上一层父级函数的局部作用域。
- 搜索变量名的优先级：**局部作用域 > 嵌套作用域 > 全局作用域**。即变量名解析机制是：在局部找不到，便会去局部外的局部找，再找不到就会去全局找。

- 如果一个函数在内部调用自身本身，这个函数就是递归函数。
- 适当地运用递归有时能够有效减少程序设计的复杂性，可以通过有限的语句来定义复杂问题，使得程序编写和维护都变得更加容易。
- 递归函数的设置一般会需要**终止条件**和**递归条件**，从而**避免无限循环**。

- 具体地，整个问题的求解可以分为两部分：第一部分是一些特殊情况，有直接的解法；第二部分与原问题相似，但比原问题的规模小，并且依赖第一部分的结果。
- 递归有两个基本要素：
 - (1) 边界条件：确定递归到何时终止，也称为**递归出口**。
 - (2) 递归模式：大问题是如何分解为小问题的，也称为**递归体**。

函数的递归调用



- 许多数学函数都是使用递归来定义的。如数字 n 的阶乘 $n!$ 可以按下面的递归方式进行定义：

$$n! = \begin{cases} n! = 1 & (n = 0) \\ n \times (n-1)! & (n > 0) \end{cases}$$

计算 $n!$ 的函数`factorial(n)`可简单地描述如下：

```
In [5]: ► def factorial (n):  
        if n==0:  
            return 1  
        return n * factorial (n - 1)  
factorial (4)
```

Out[5]: 24

● 函数的递归调用

- 利用递归方法计算斐波那契数列，其具有如下性质

$$\text{fib}(0)=0 \quad n=0$$

$$\text{fib}(1)=1 \quad n=1$$

$$\text{fib}(n)=\text{fib}(n-1)+\text{fib}(n-2) \quad n>2$$

```
def fibonacci(n):  
    if n <= 0:  
        return "输入错误: n 应为正整数"  
    elif n == 1:  
        return 0 # 根据定义, 斐波那契数列第1项为0  
    elif n == 2:  
        return 1 # 斐波那契数列第2项为1  
    else:  
        return fibonacci(n-1) + fibonacci(n-2)  
  
# 测试代码  
n = 4  
print(f"斐波那契数列第{n}项的值是: {fibonacci(n)}")
```

斐波那契数列第4项的值是: 2

● 总结

- 函数定义时建立的参数为形式参数
- 函数实际使用时输入的参数就是实参
- 变量的作用域反映变量在不同程序结构层次内的有效性
- **lambda**表达式用来定义匿名函数
- 函数递归是一种常见的编程方式